CNENGR 116
Fundamentals of Digital Electronics

Lecture 8 – Finite State Machines

Dr Zhongming Zhang

Zhongming.zhang@lancaster.ac.uk

School of Engineering

Lancaster University

Science and Technology at Lancaster

Lancaster University College at Beijing Jiaotong University

18:38 12月2日周四 37%

$$AB + AC + B'C$$

$$= AB + \bar{B}C + (B+\bar{B})AC$$

$$= AB + \bar{B}C + ABC + A\bar{B}C$$

$$= ABC(1+C) + \bar{B}C(1+A)$$

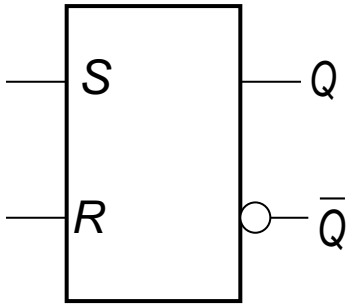
$$= AB + \bar{B}C$$

And Q2 (b)….

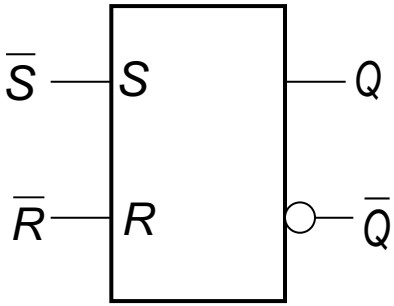
Recap

- The SR Latch – basic memory element – 1 illegal state
- SR Latch with Enable input
- Latch is LEVEL triggered; a Flip flop is EDGE triggered
- D-type built from the RESET being the complement of SET – i.e. no possibility of Set and Reset at the same time
- Can add PRESET and CLEAR that are asynchronous – i.e. Do not depend on the state of the clock / trigger.

Logic Symbols for Latches

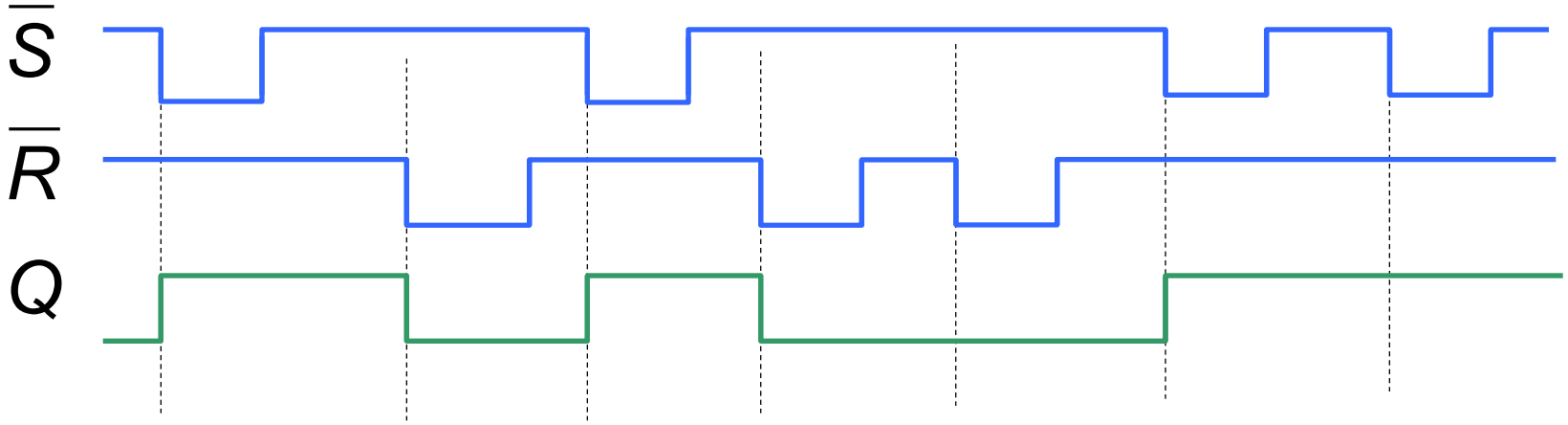


Active-HIGH input
S-R latch

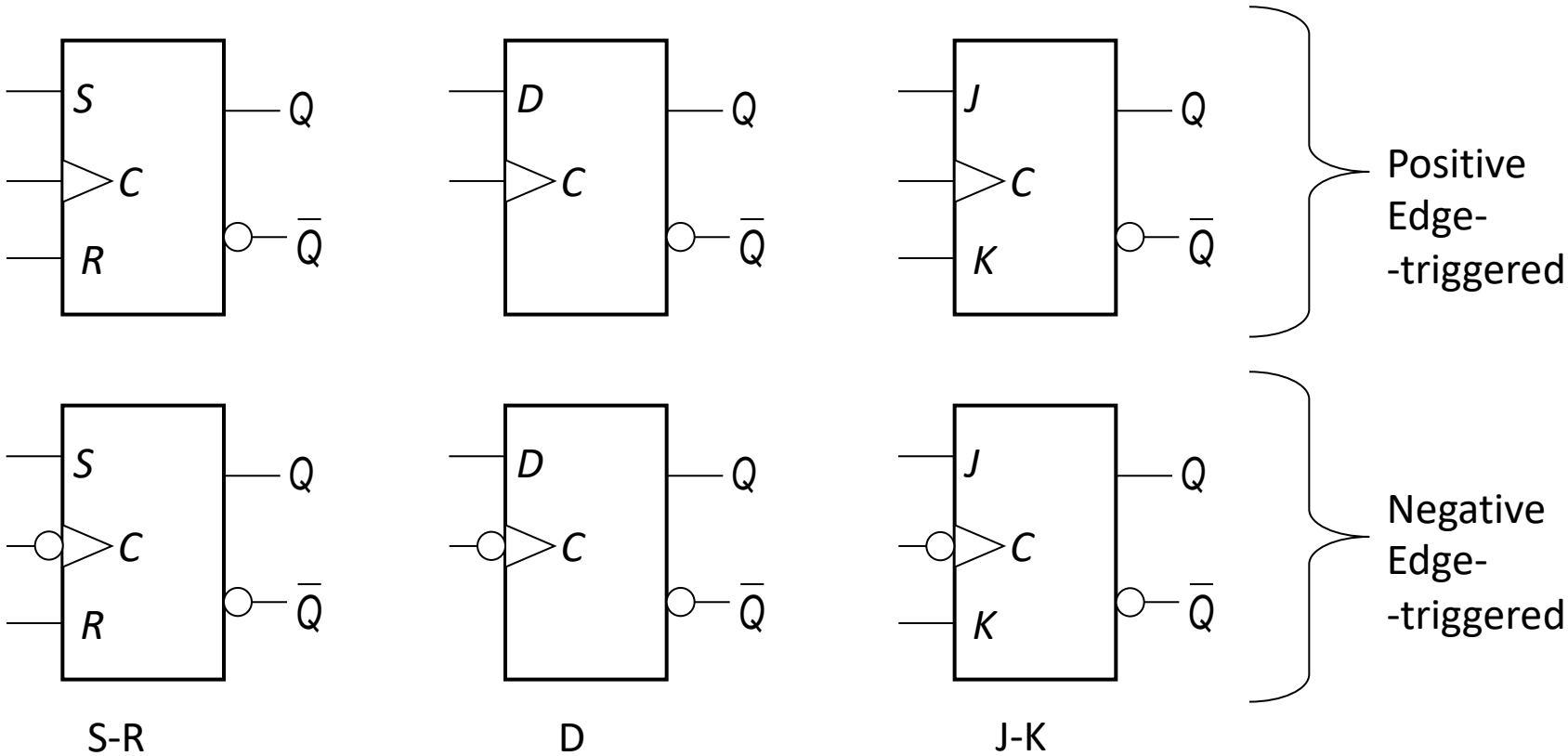


Active-LOW input
$\overline{S}-\overline{R}$ latch

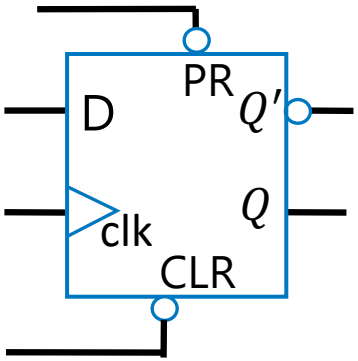
Time flow diagram for the active-LOW input $\overline{S}-\overline{R}$ latch:

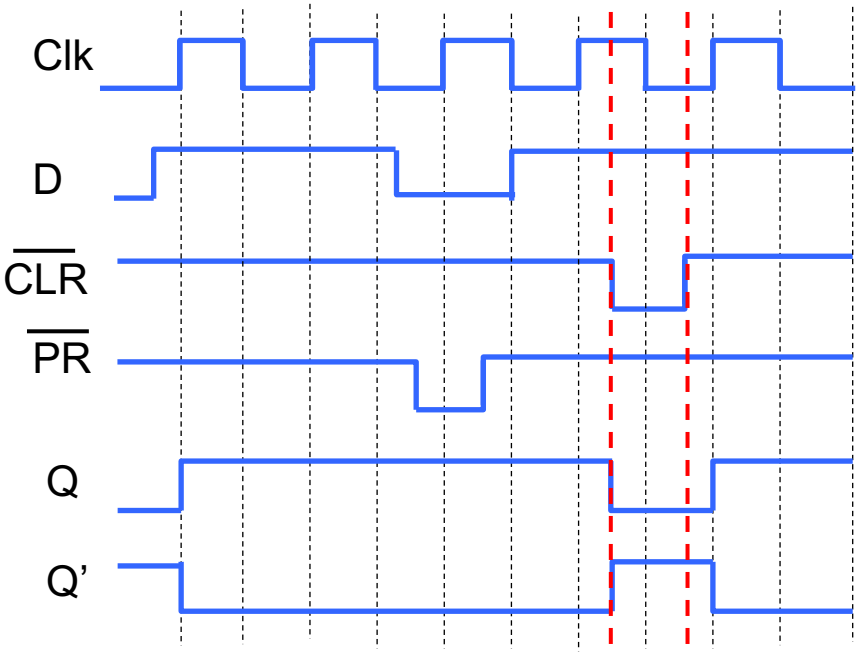


Edge-triggered Flip-Flops



D-Type Flip Flop with Preset & Clear





Objectives

- Understanding the concept of state machines
- Design and analysis of the behaviour of state machines
- Familiarity with different types of state machines (Mealy and Moore)

1. Finite State Machine Concept

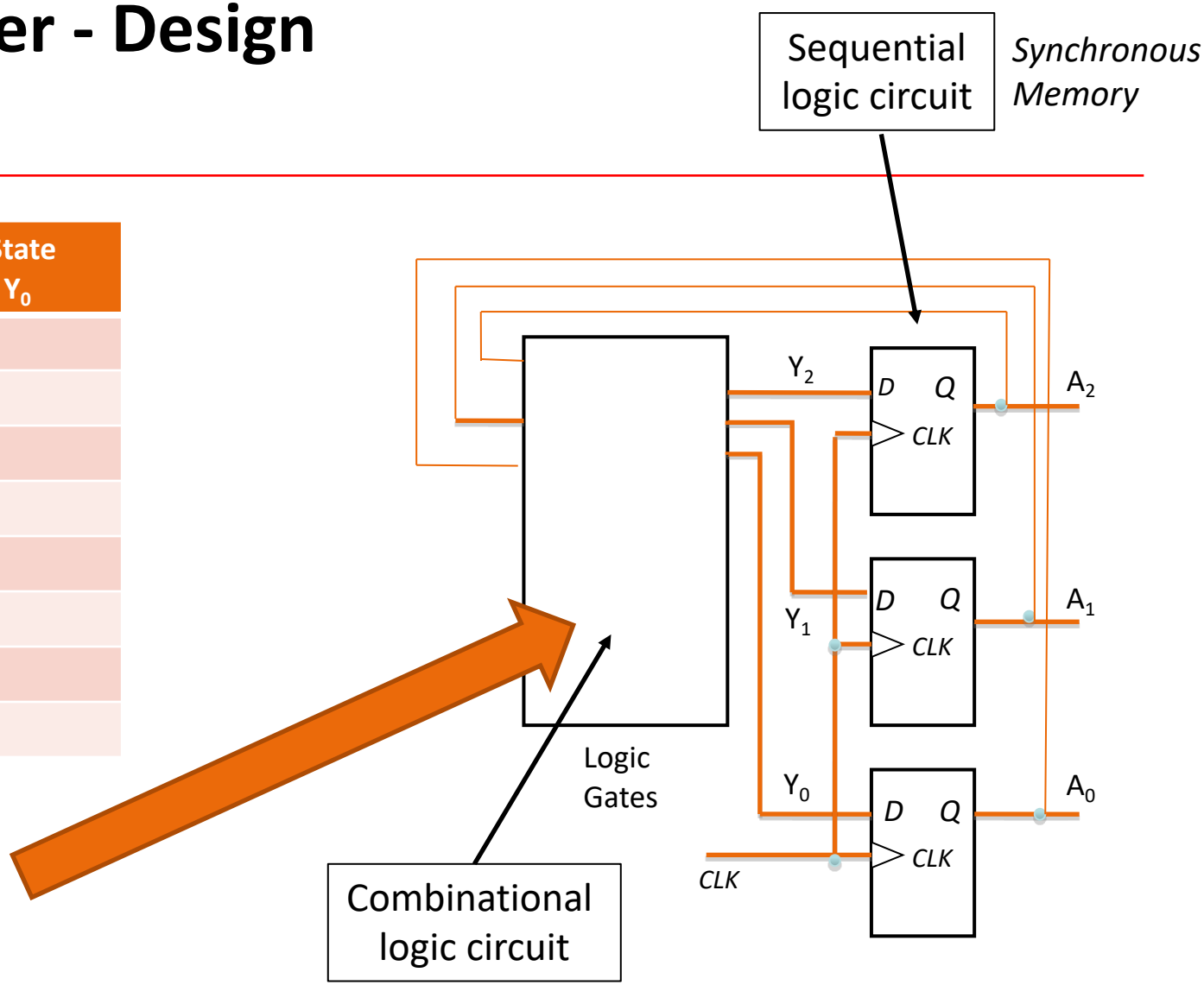
- Generates a change in response to a clock pulse based on the current state of the system and in most cases a set of input conditions
  - Example – a counter
    - No inputs
    - Transitions in response to a clock
    - Cyclic

Current State A_2, A_1, A_0	Next State Y_2, Y_1, Y_0
000	001
001	010
010	011
011	100
100	101
101	110
110	111
111	000

3 Bit Counter - Design

Current State A_2, A_1, A_0	Next State Y_2, Y_1, Y_0
000	001
001	010
010	011
011	100
100	101
101	110
110	111
111	000

$Y_2=f(A_2, A_1, A_0)$
$Y_1=f(A_2, A_1, A_0)$
$Y_0=f(A_2, A_1, A_0)$



3 Bit Counter

Current State A_2, A_1, A_0	Next State Y_2, Y_1, Y_0
000	001
001	010
010	011
011	100
100	101
101	110
110	111
111	000

$$\boldsymbol{Y_0 = \overline{A_0}}$$

$$Y_1 = \overline{A_2}\,\overline{A_1}A_0 + \overline{A_2}A_1\overline{A_0} + A_2\overline{A_1}A_0 + A_2A_1\overline{A_0}$$
$$Y_1 = A_1\overline{A_0}(\overline{A_2} + A_2) + \overline{A_1}A_0(\overline{A_2} + A_2)$$
$$\boldsymbol{Y_1 = A_1 \oplus A_0}$$

$$\boldsymbol{Y_2 = A_2\overline{A_1} + A_2\overline{A_0} + \overline{A_2}A_1A_0}$$

Calculate the outputs from the logic required to define the next state

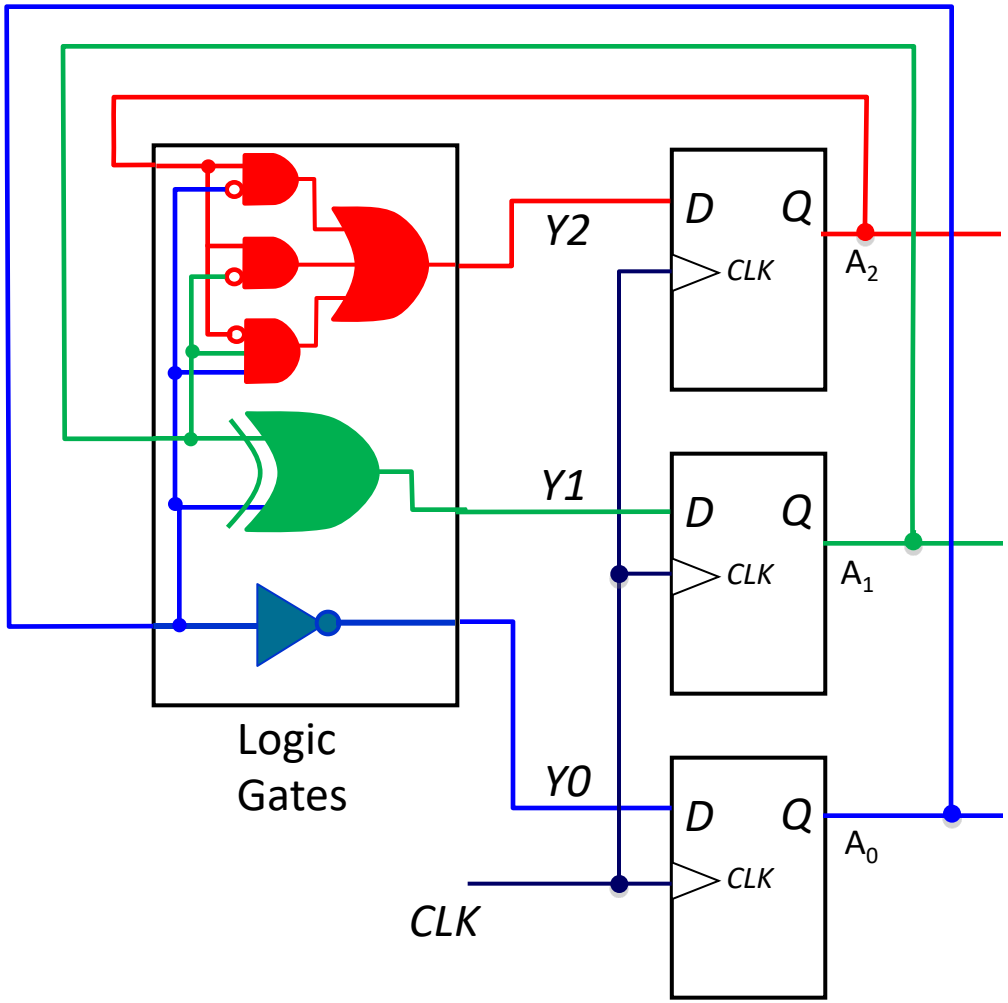
FSM 3 Bit Counter

$$\boldsymbol{Y_0 = \overline{A_0}}$$

$$Y_1 = \overline{A_2}\,\overline{A_1}A_0 + \overline{A_2}A_1\overline{A_0} + A_2\overline{A_1}A_0 + A_2A_1\overline{A_0}$$
$$Y_1 = A_1\overline{A_0}(\overline{A_2} + A_2) + \overline{A_1}A_0(\overline{A_2} + A_2)$$
$$\boldsymbol{Y_1 = A_1 \oplus A_0}$$

$$\boldsymbol{Y_2 = A_2\overline{A_1} + A_2\overline{A_0} + \overline{A_2}A_1A_0}$$

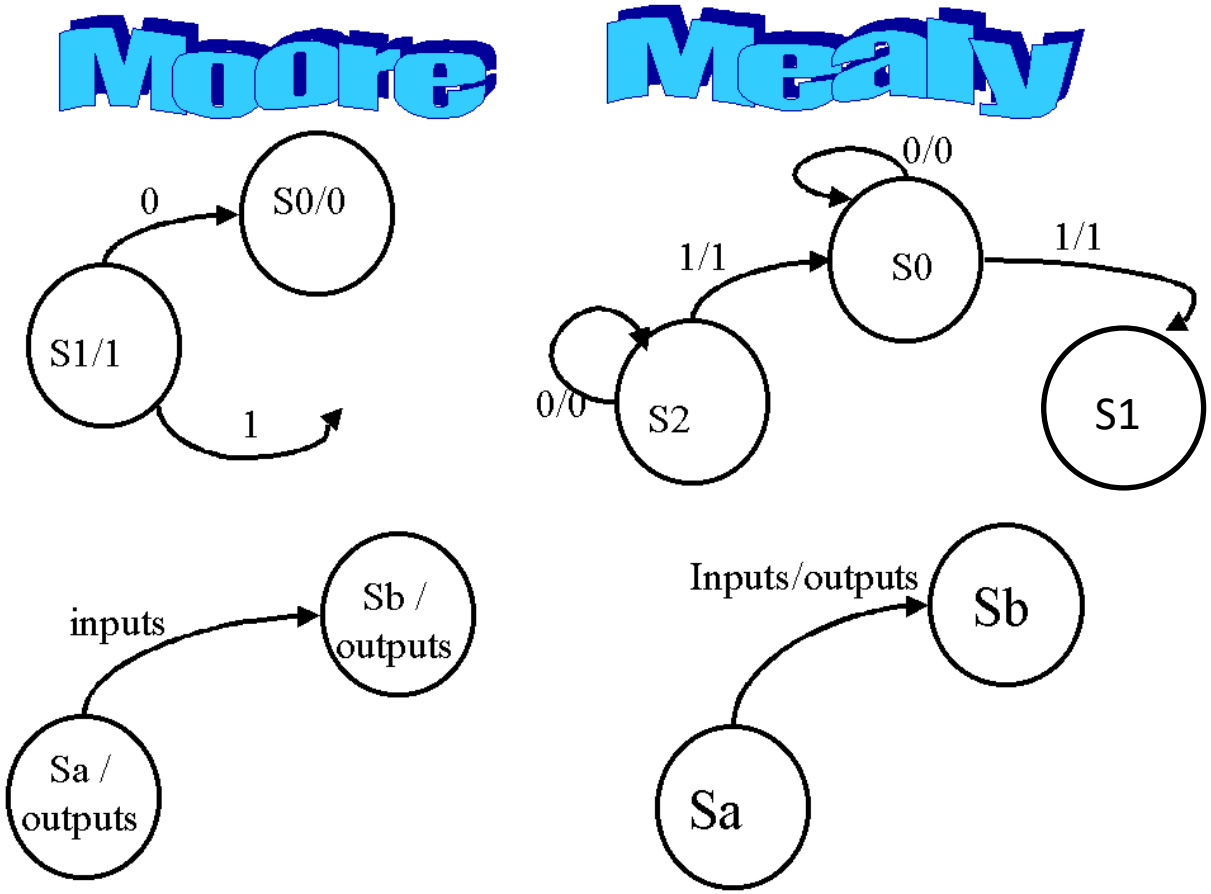
This design will now count on every pulse of the clock



1.1. Finite State Machines Definitions

- **Inputs**
  - Inputs are any form of stimulus from the outside world
  - Often a switch or a binary sensor
  - FSMs are low level, this means most inputs will be single bits
- **State**
  - Describes the current condition of the finite state machine
  - The state of a FSM is determined by its history
- **Outputs**
  - Outputs are linked to the FSM's state but are not the same thing
  - Outputs can depend either on the state only (Moore) or on the state and inputs (Mealy)

1.1. Mealy and Moore Machines



1.2. Turnstile – A Practical Example

- A turnstile used to control access to subways, …
  - Initially the arms are locked
  - Depositing a coin in a slot on the turnstile unlocks the arms
  - Pushing and passing through the arm returns it to locked
- Thinking of its operation in terms of a **transition table**

Present state	Inputs	Next state	Outputs
Locked	Coin		Green arrow
	Push		Red cross
Unlocked	Coin		Green arrow
	Push		Red cross

2. Finite State Machine Design Process

- **Two steps**
  - A way to **capture** desired sequential behaviour
  - A way to **convert** such a behaviour to a sequential circuit

This is like the approach we had for designing combinational circuits

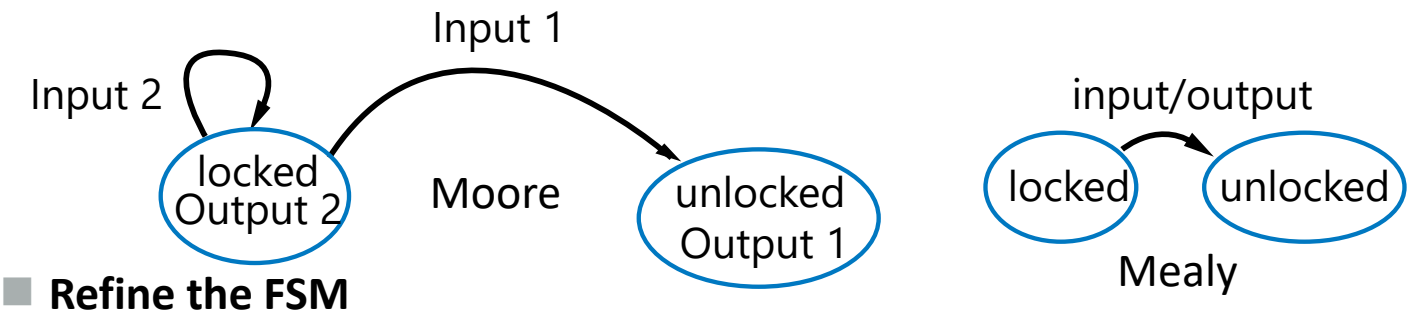
Step		Description
Step 1: Capture behavior	**Capture** the function	Create a truth table or equations, ***whichever is most natural for the given problem***, to describe the desired behavior of each output of the combinational logic.
Step 2: Convert to circuit	2A: **Create** equations	This substep is only necessary if you captured the function using a truth table instead of equations. Create an equation for each output by ORing all the minterms for that output. Simplify the equations if desired.
	2B: **Implement** as a gate-based circuit	For each output, create a circuit corresponding to the output's equation. (Sharing gates among multiple outputs is OK optionally.)

2.1. Capturing the Behaviour

- Finite State Machine (FSM) can be used to capture the sequential behaviour as follows:
  - **List states**
    - Give meaningful names, show initial state
    - Optionally add some transitions if they help
    - A <u>circle</u> with the state name in it for each state

2.1. Capturing the Behaviour

- **Create transitions**
  - For each state, define all possible transitions leaving that state.
  - A directed arc from the present state to the next state for each state transition.
  - A label on each directed arc/circle with the Input/output values which causes the state transition or produced from that state.



- **Refine the FSM**
  - Execute the FSM mentally and make any needed improvements.

2.2. A Quick Exercise

- Draw the Moore state diagram for the turnstile example reviewed in previous slides.

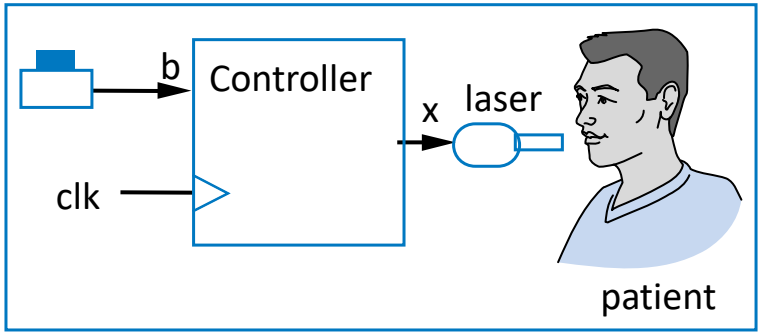
Present state	Inputs	Next state	Outputs
Locked	Coin	Unlocked	Green arrow
	Push	Locked	Red cross
Unlocked	Coin	Unlocked	Green arrow
	Push	Locked	Red cross

State Transition Diagrams & Tables

NOT AVAILABLE IN PREVIEW

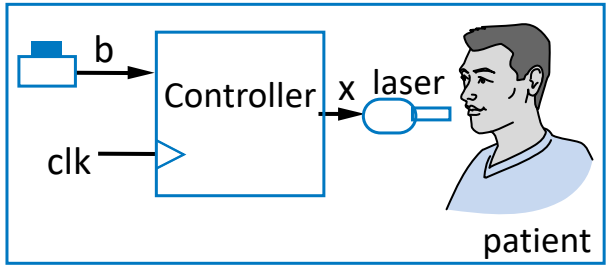
2.3. Converting FSM to the Circuit

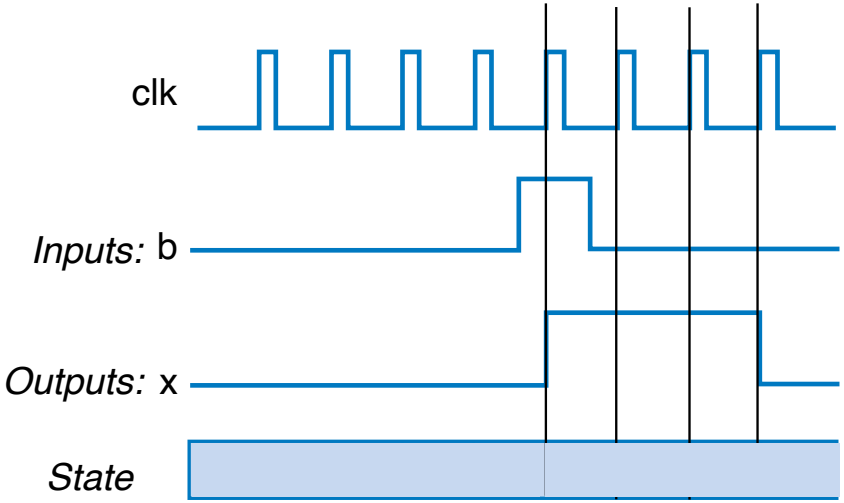
- **Designing a laser surgery system**
  - Turning on the laser for precise amount of time (30 ns)
  - Assume b stays high for exactly 1 clock cycle
  - System's clock period is 10 ns
  - Pushing the button b (b=1) will turn the output on (x=1) for exactly 3 clock cycles



2.3. Converting FSM to the Circuit

- **Capturing the behaviour-Timing diagram**
  - Set of inputs: {b}
  - Set of outputs: {x}
  - Initial state: “Off”
  - Timing diagram is an effective way to capture the states of a time ordered behaviours

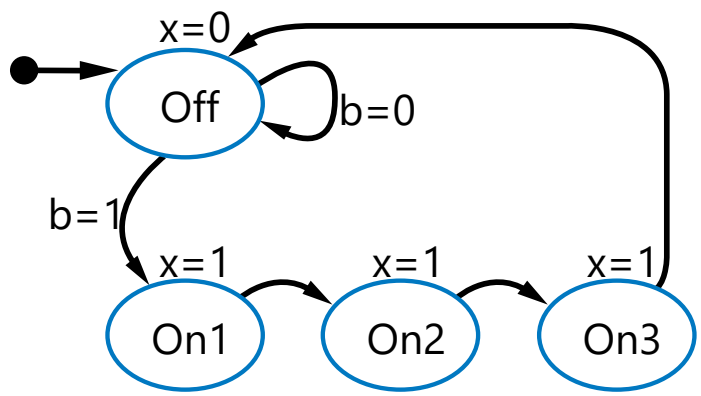




2.3. Converting FSM to the Circuit

- **Capturing the behaviour-State diagram**
  - Set of actions corresponds to the set of outputs in each state.
  - Initially x=0, then x=1 for the first time, x=1 for the second time, and x=1 for the third time.
  - Set of states: {Off, On1, On2, On3}.
  - State diagram is used to capture the transitions between the states

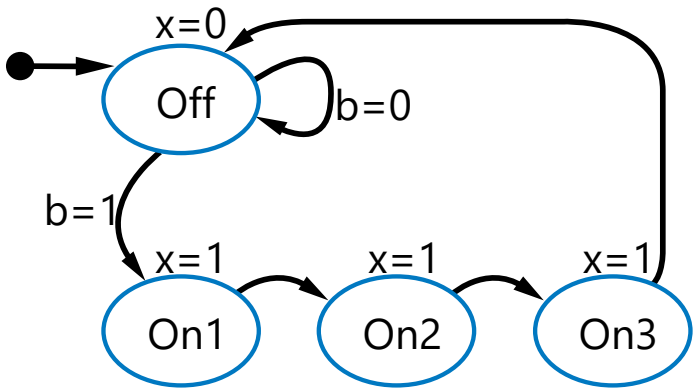
Inputs: b; Outputs: x



2.3. Converting FSM to the Circuit

- **Capturing the behaviour-Transition table**
  - State transition diagram can be converted to the transition table and vice versa

Present state	Input b	Next state	Output x
off			
on1			
on2			
on3			



Our states encoded;
Off:**00**, On1:**01**, On2:**10**, On3:**11**

2.3. Converting FSM to the Circuit

- Fill in the truth table and convert it to the equation

Inputs			Outputs		
s1	s0	b	n1	n0	x
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	1	0	1
0	1	1	1	0	1
1	0	0	1	1	1
1	0	1	1	1	1
1	1	0	0	0	1
1	1	1	0	0	1

Present state s1 s0	Inputs b	Next state n1 n0	Outputs x
0 0	0	0 0	0
	1	0 1	0
0 1	x	1 0	1
1 0	x	1 1	1
1 1	x	0 0	1

$$x = s1 + s0$$

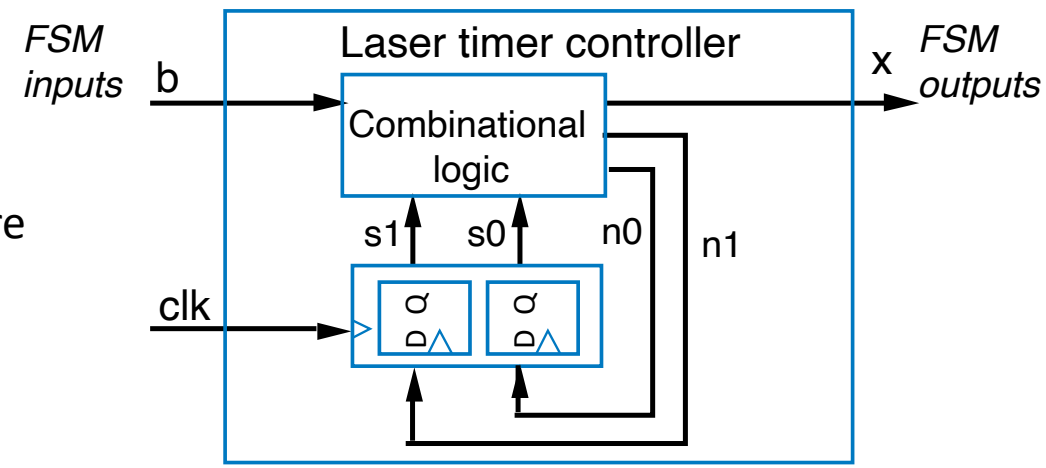
$$n1 = \overline{s1}s0 + s1\overline{s0}$$

$$n0 = \overline{s1}\,\overline{s0}b + s1\overline{s0}$$

2.3. Converting FSM to the Circuit

- Controller design is the process of converting FSM to a sequential circuit
  - Flip-flops store encoding of current state e.g., Off**:00**, On1**:01**, On2**:10**, On3**:11**
  - Combinational logic computes outputs and next state from inputs and current state
  - Rising clock edge takes the controller to the next state

Standard controller architecture



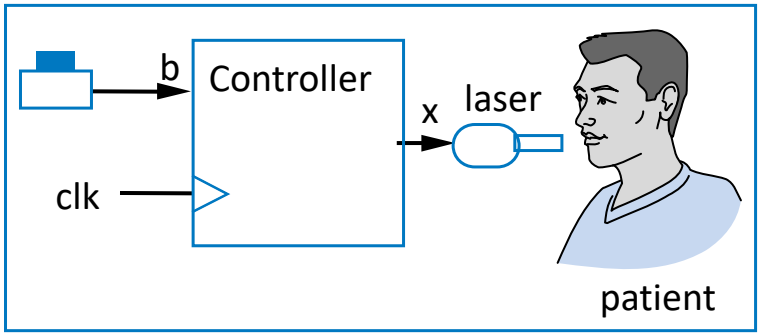
2.3. Converting FSM to the Circuit

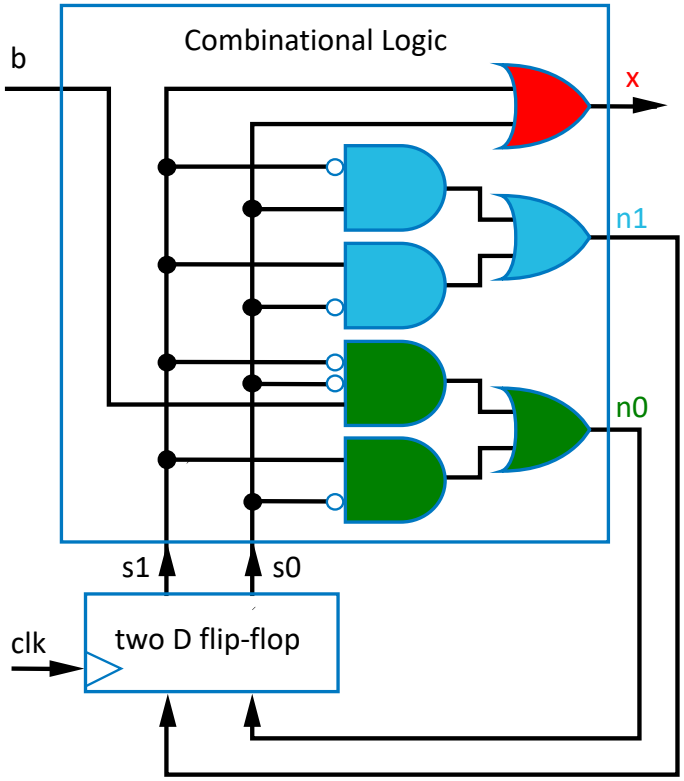
- Controller is obtained by converting equations to the circuit

$$x = s1 + s0$$

$$n1 = \overline{s1}s0 + s1\overline{s0}$$

$$n0 = \overline{s1}\,\overline{s0}b + s1\overline{s0}$$





2.3. Controller Design Process-Summary

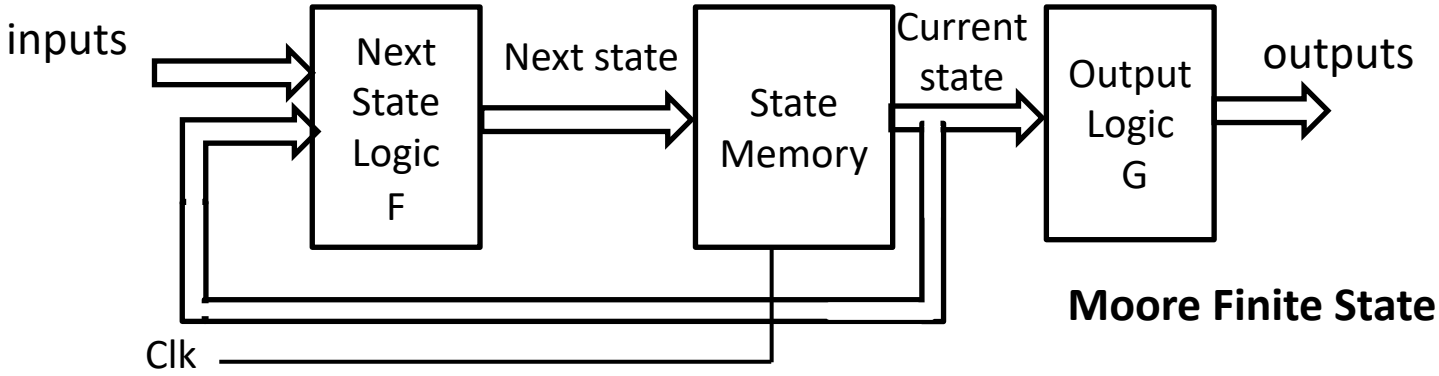
Step		Description
Step 1: Capture behaviour	**Capture** the FSM	Create an FSM that describes the desired behaviour of the controller.
Step 2: Convert to circuit	2A: **Set up** architecture	Use state register of appropriate width and combinational logic. The logic's inputs are the state register bits and the FSM inputs; outputs are next state bits and the FSM outputs.
	2B: **Encode** the states	Assign unique binary number (encoding) to each state. Usually use fewest bits, assign encoding to each state by counting up in binary.
	2C: **Fill in** the truth table	Translate FSM to truth table for combinational logic such that the logic will generate the outputs and next state signals for the given FSM. Ordering the inputs with state bits first makes the correspondence between the table and the FSM clear.
	2D: **Implement** combinational logic	Implement the combinational logic using any method.

3. Mealy and Moore Machines

- **Clocked Synchronous State Machines**
  - **Characteristic equation** specifies the next state of a machine as a function of its current state
  - The next state of the FSM is determined by the logic function F

    next state = F (current state, input)
  - The output logic G determines the output of FSM as

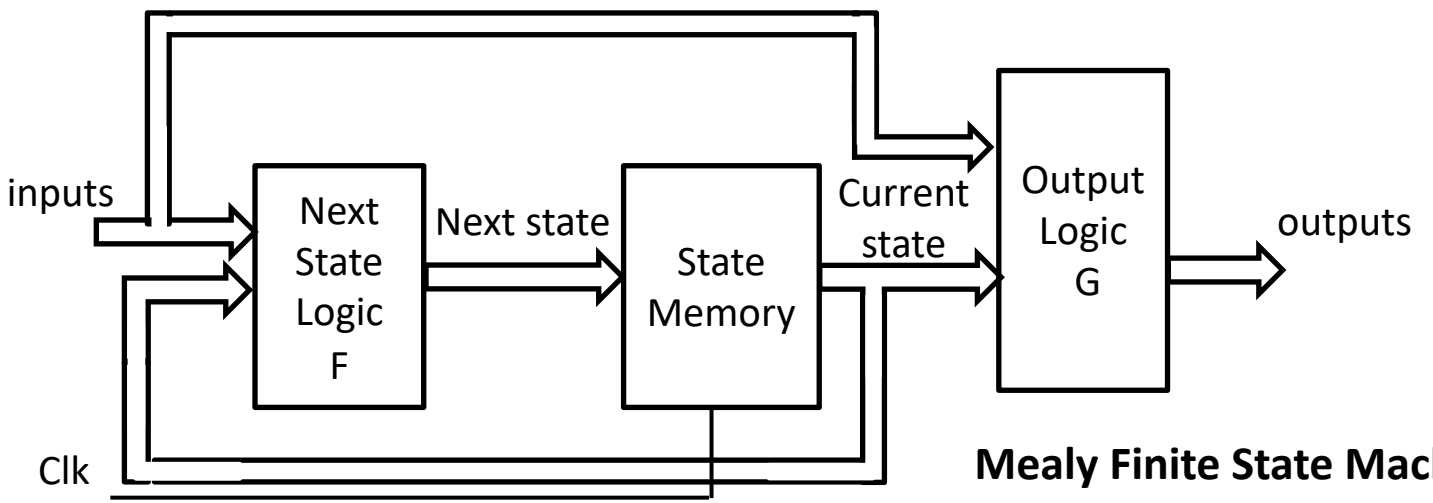
    Output = G (state), or **Output = G (current state, input)**



Moore Finite State Machine

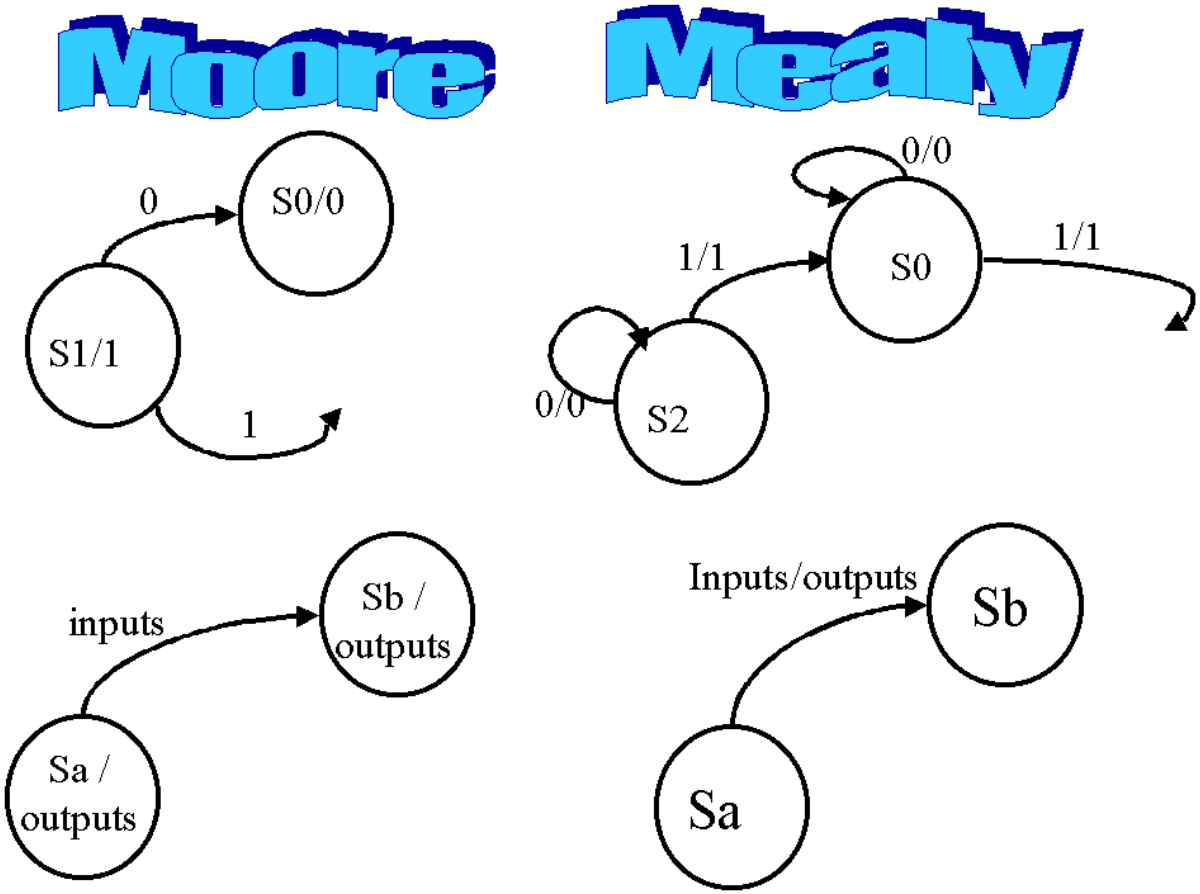
3. Mealy and Moore Machines

- Memory is made of flip-flops and they change the states at each **tick** of the clock.
- If the output of the state machine depends only on the states it is denoted as **Moore machine**.
- If the output of the state machine depends both on the states and input it is denoted as **Mealy machine**.



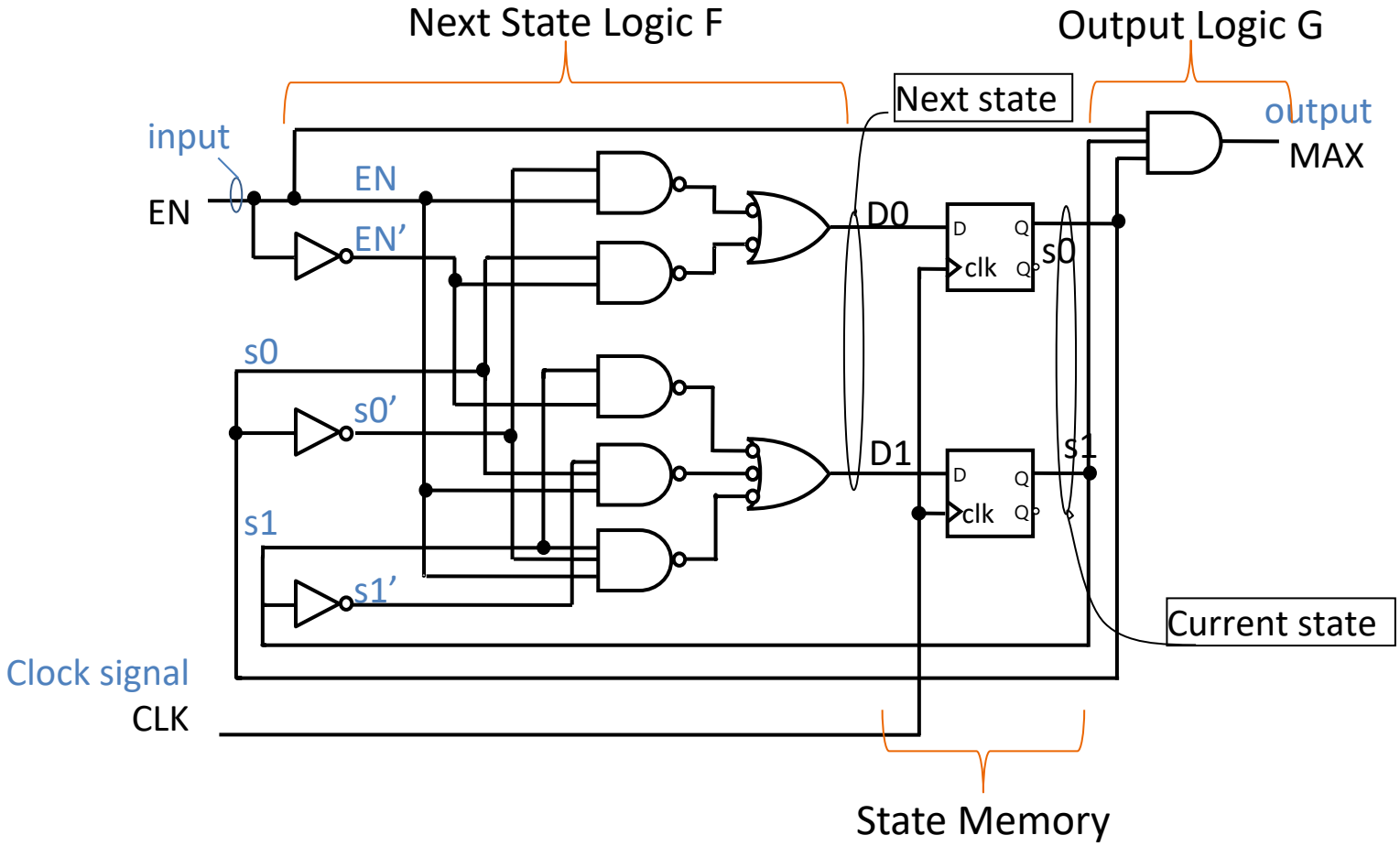
Mealy Finite State Machine

3. Mealy and Moore Machines



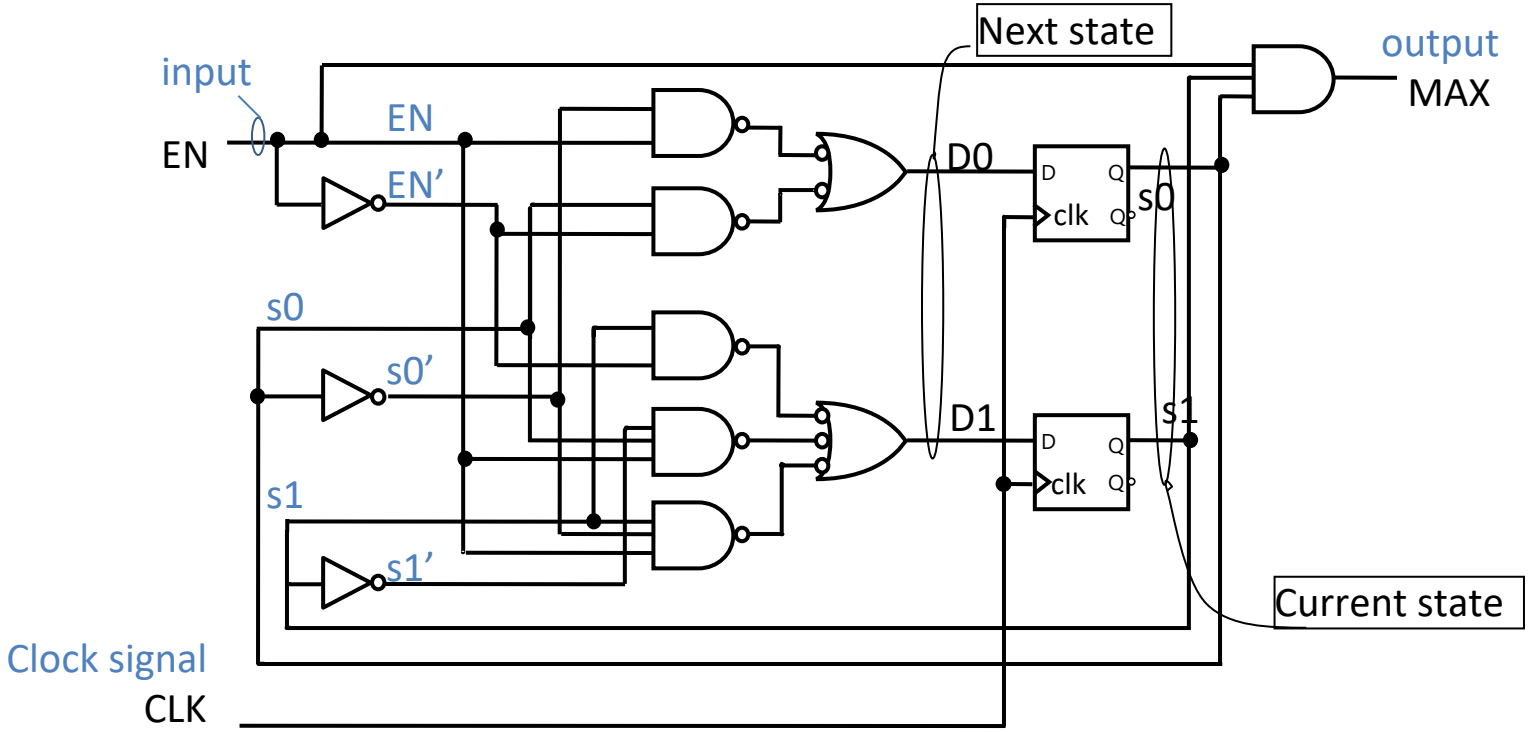
3.1. A Quick Exercise

- A) The inputs and outputs of the 2-bit counter circuit are shown here. Determine the logic blocks representing the **memory** and functions **F** and **G**.



3.1. A Quick Exercise

- B) Is it representing a Mealy or Moore machine?



3.1. A Quick Exercise

- C) Given the transition table of the state machine as below draw the corresponding state diagram.

NOT AVAILABLE IN PREVIEW

Transition table

Present state		Next state/Output	
		EN=0	EN=1
s1	s0	n1n0	n1n0
0	0	0 0, 0	0 1, 0
0	1	0 1, 0	1 0, 0
1	0	1 0, 0	1 1, 0
1	1	1 1, 0	0 0, 1

Summary

- Introduced the concept of finite state machine
- How to capture a sequential behaviour and convert it to the circuit
- Introduced two types of state machines called Mealy and Moore